# GATE DA 2027

## Data Structures

# Linked Lists, Trees & Hash Tables

**Complete Teaching and Practice Notes**

Theory + Properties + Visualizations + Numericals
GATE PYQ Concepts + Practice Questions + Detailed Solutions

| | |
|---|---|
| **Exam** | GATE Data Science and Artificial Intelligence |
| **Target** | GATE DA 2027 |
| **Topics** | Linked Lists, Trees, Hash Tables |
| **Level** | Basic → Advanced |
| **Purpose** | Classroom Teaching + Revision + Practice |

**Teaching Philosophy**

Draw the structure → Trace the operation → Find the invariant → Calculate complexity.

# Contents

**1 Introduction to Data Structures** **4**
1.1 Four Questions to Ask . . . 4

**2 Linked Lists** **5**
2.1 Basic Node . . . 5
2.2 Advantages of Linked Lists . . . 5
2.3 Disadvantages . . . 5
2.4 Types of Linked Lists . . . 6
2.4.1 Singly Linked List . . . 6
2.4.2 Doubly Linked List . . . 6
2.4.3 Circular Linked List . . . 6
2.5 Complexity of Linked List Operations . . . 6

**3 Linked List Insertion** **7**
3.1 Insertion at Beginning . . . 7
3.2 Insertion After a Given Node . . . 7
3.3 Insertion at End . . . 8

**4 Linked List Deletion** **9**
4.1 Delete First Node . . . 9
4.2 Delete After a Known Node . . . 9
4.3 Delete Last Node . . . 9

**5 Linked List Reversal** **10**

**6 Linked List Fast–Slow Pointer Technique** **11**
6.1 Cycle Detection . . . 11

**7 Linked List Numericals** **11**

**8 Linked List Practice Questions** **13**

**9 Trees** **15**
9.1 Important Terminology . . . 15

**10 Binary Trees** **16**
10.1 Maximum Nodes at Level . . . 16
10.2 Minimum Nodes . . . 16
10.3 Important Binary Tree Property . . . 16

**11 Binary Tree Traversals** **18**
11.1 Preorder . . . 18

11.2 Inorder . . . 18
11.3 Postorder . . . 18
11.4 Level Order . . . 18

**12 Tree Reconstruction** **20**
12.1 Preorder + Inorder . . . 20

**13 Tree Practice** **21**

**14 Binary Search Trees** **22**
14.1 BST Search . . . 22

**15 BST Insertion and Deletion** **24**
15.1 Insertion . . . 24
15.2 BST Deletion . . . 24
15.2.1 Case 1: Leaf . . . 24
15.2.2 Case 2: One Child . . . 24
15.2.3 Case 3: Two Children . . . 24

**16 AVL Trees** **26**
16.1 Four Rotation Cases . . . 26
16.2 LL Case . . . 26
16.3 RR Case . . . 27
16.4 LR Case . . . 27
16.5 RL Case . . . 27

**17 AVL Complexity** **28**

**18 Heaps** **29**
18.1 Max Heap . . . 29
18.2 Min Heap . . . 29
18.3 Heap Array Representation . . . 29
18.4 Heap Complexity . . . 30

**19 Heap Practice** **31**

**20 Hash Tables** **32**
20.1 Hash Function . . . 32
20.2 Collision . . . 32

**21 Load Factor** **32**

**22 Collision Resolution** **34**
22.1 Separate Chaining . . . 34

22.2 Open Addressing . . . . . 34

**23 Linear Probing** **35**

23.1 Primary Clustering . . . . . 36

# 1 Introduction to Data Structures

A data structure is a systematic way of organizing and storing data so that operations on the data can be performed efficiently.

For GATE DA, the important questions are usually based on:

- representation of data structures,
- operations,
- complexity analysis,
- tracing,
- structural properties,
- counting,
- implementation details.

**GATE Focus**

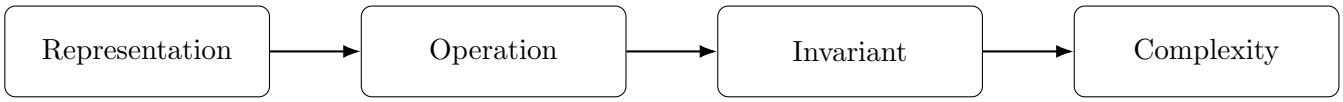
For this chapter, the three most important structures are:

1. Linked Lists
2. Trees
3. Hash Tables

## 1.1 Four Questions to Ask

Whenever you see a data structure question, ask:

1. How is the data represented?
2. What operation is being performed?
3. What invariant/property is maintained?
4. What is the time and space complexity?



# 2 Linked Lists

A linked list is a linear data structure in which elements are stored in nodes and nodes are connected using references or pointers.

Unlike an array, the nodes of a linked list need not occupy contiguous memory locations.

## 2.1 Basic Node

For a singly linked list:

$$\boxed{\text{Node} = \text{Data} + \text{Next Pointer}}$$




The first node is accessed using the `head` pointer.

$$head \rightarrow 10 \rightarrow 25 \rightarrow 40 \rightarrow NULL$$

**Important**

For an acyclic singly linked list containing $n$ nodes, exactly $n-1$ `next` pointers point to another node and one `next` pointer contains NULL.

## 2.2 Advantages of Linked Lists

- Dynamic size.
- Efficient insertion/deletion when the required node/position is known.
- No requirement for contiguous memory.
- Useful for implementing stacks, queues and other structures.

## 2.3 Disadvantages

- No direct/random access.
- Extra memory is required for pointers.
- Poor cache locality compared with arrays.
- Traversal is sequential.

## 2.4 Types of Linked Lists

There are three major types:

1. Singly linked list
2. Doubly linked list
3. Circular linked list

### 2.4.1 Singly Linked List

$$A \to B \to C \to NULL$$

Each node contains one link.

### 2.4.2 Doubly Linked List

$$NULL \leftarrow A \leftrightarrow B \leftrightarrow C \to NULL$$

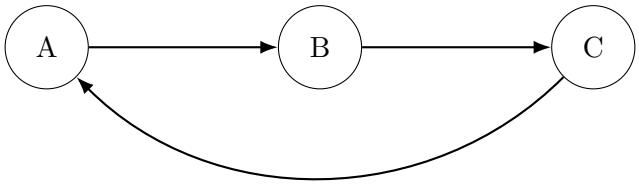
Each node contains:

$$\boxed{prev + data + next}$$

### 2.4.3 Circular Linked List

$$A \to B \to C \to A$$

The last node points back to the first node.




## 2.5 Complexity of Linked List Operations

| Operation | Singly Linked List | Array |
|---|---|---|
| Access by index | $O(n)$ | $O(1)$ |
| Search | $O(n)$ | $O(n)$ |
| Insert at beginning | $O(1)$ | $O(n)$ |
| Delete at beginning | $O(1)$ | $O(n)$ |
| Insert after known node | $O(1)$ | $O(n)$ |
| Delete after known node | $O(1)$ | $O(n)$ |

**Common GATE Trap**

Do not say "deletion in a linked list is always $O(1)$."
If the node must first be searched for, the total complexity is usually $O(n)$.

# 3 Linked List Insertion

## 3.1 Insertion at Beginning

Suppose:

$$head \to A \to B \to C$$

We want to insert $X$.

First:

$$X.next = head$$

Then:

$$head = X$$

Final:

$$head \to X \to A \to B \to C$$

$$\boxed{O(1)}$$

## 3.2 Insertion After a Given Node

Suppose:

$$A \to B \to C$$

Insert $X$ after $B$.

Correct order:

$$X.next = B.next$$

then

$$B.next = X$$

Final:

$$A \to B \to X \to C$$

**Common GATE Trap**

If you change $B.next$ before saving the old successor, the rest of the list may become inaccessible.

### 3.3 Insertion at End

If only the head pointer is available:

$$\boxed{O(n)}$$

because the last node must be found.

If a tail pointer is maintained:

$$\boxed{O(1)}$$

because insertion can be performed directly after the tail.

# 4 Linked List Deletion

## 4.1 Delete First Node

If:

$$head \to A \to B \to C$$

then:

$$head = head.next$$

Final:

$$head \to B \to C$$

Complexity:

$$\boxed{O(1)}$$

## 4.2 Delete After a Known Node

Suppose:

$$A \to B \to C$$

Delete $B$.
Set:

$$A.next = B.next$$

Final:

$$A \to C$$

Complexity:

$$\boxed{O(1)}$$

provided $A$ is already known.

## 4.3 Delete Last Node

In a singly linked list, to delete the last node we normally need the second-last node.
Therefore:

$$\boxed{O(n)}$$

unless additional information is available.

## 5 Linked List Reversal

This is one of the most important linked-list algorithms.

Maintain three pointers:

$$prev, \quad curr, \quad next$$

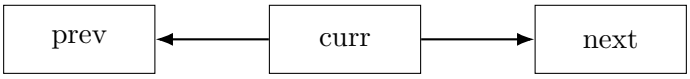
For each node:

$$next = curr.next$$

$$curr.next = prev$$

$$prev = curr$$

$$curr = next$$




Time:

$$\boxed{O(n)}$$

Extra space:

$$\boxed{O(1)}$$

**Important**

Always save `curr.next` before changing `curr.next`.

# 6 Linked List Fast–Slow Pointer Technique

Use two pointers:

$$slow \rightarrow \text{one step}$$

$$fast \rightarrow \text{two steps}$$

Applications:

- Finding middle of linked list.
- Detecting cycles.
- Finding cycle entry under suitable algorithm.

## 6.1 Cycle Detection

If:

$$slow = slow.next$$

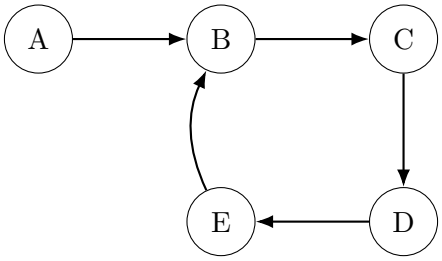
and

$$fast = fast.next.next$$

then:

$$slow = fast$$

at some point if a cycle exists.




# 7 Linked List Numericals

**Practice 1 – Numerical**

A singly linked list contains 37 nodes. How many `next` pointers contain non-NULL values?

**Detailed Solution**

For $n$ nodes in an acyclic singly linked list:

$$\text{non-NULL next pointers} = n - 1$$

Therefore:

$$37 - 1 = \boxed{36}$$

**Practice 2 – Numerical**

A doubly linked list contains 20 nodes. Ignoring external head and tail variables, how many pointer fields are present?

**Detailed Solution**

Each node contains two pointer fields:

$$prev,\ next$$

Therefore:

$$20 \times 2 = \boxed{40}$$

**Practice 3 – Numerical**

A singly linked list contains 50 nodes. How many links must be followed to reach the 50th node from the head?

**Detailed Solution**

To reach node $k$:

$$k - 1$$

links are followed.
Therefore:

$$50 - 1 = \boxed{49}$$

**Practice 4 – Numerical**

A node $P$ is already known in a linked list. A new node $X$ is inserted immediately after $P$. How many pointer assignments are required?

**Detailed Solution**

$$X.next = P.next$$

$$P.next = X$$

Hence:

$$\boxed{2}$$

# 8 Linked List Practice Questions

**Practice 5 – MCQ**

Which operation is $O(1)$ in a singly linked list when the head pointer is available?

1. Access the 10th element
2. Search for an element
3. Insert at beginning
4. Access the last element

**Answer**

C

**Practice 6 – MCQ**

Which technique is commonly used to detect a cycle in a linked list?

1. Binary search
2. Floyd's fast–slow pointer algorithm
3. Heapify
4. Hashing only

**Answer**

B

**Practice 7 – NAT**

Two sorted linked lists contain 17 and 23 nodes. What is the maximum number of comparisons needed by the standard merge algorithm?

**Detailed Solution**

Maximum comparisons:

$$m + n - 1$$

Therefore:

$$17 + 23 - 1 = \boxed{39}$$

**Practice 8 – NAT**

A linked list contains 64 nodes. During in-place reversal, how many times is the assignment

$$curr.next = prev$$

executed?

**Answer**

Once for each node:

$$\boxed{64}$$

# 9 Trees

A tree is a connected acyclic graph.

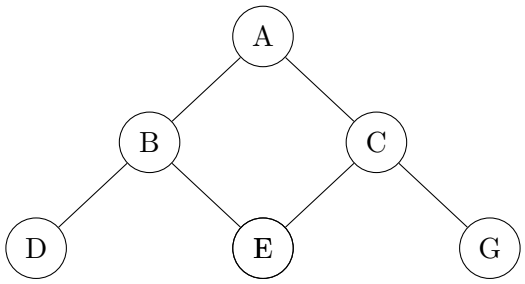
For a tree containing $n$ vertices:

$$\boxed{E = n - 1}$$

where $E$ is the number of edges.

## 9.1 Important Terminology

- Root
- Parent
- Child
- Sibling
- Leaf
- Internal node
- Degree
- Depth
- Height
- Subtree




For the above tree:

- Root = A
- Leaves = D,E,F,G
- Internal nodes = A,B,C
- Depth of A = 0
- Height = 2 edges

**Important**

Always check whether the question defines height in terms of edges or number of levels/nodes.

# 10 Binary Trees

A binary tree is a tree in which each node has at most two children.

They are called:

$$\boxed{\text{Left Child and Right Child}}$$

## 10.1 Maximum Nodes at Level

At level $i$:

$$\boxed{2^i}$$

Therefore, maximum nodes in a binary tree of height $h$:

$$1 + 2 + 4 + \cdots + 2^h$$

Using geometric progression:

$$\boxed{2^{h+1} - 1}$$

## 10.2 Minimum Nodes

For height $h$, a completely skewed tree has:

$$\boxed{h + 1}$$

nodes.

Therefore:

$$\boxed{h + 1 \leq n \leq 2^{h+1} - 1}$$

## 10.3 Important Binary Tree Property

Let:

$$n_0 = \text{number of nodes with 0 children}$$

$$n_1 = \text{number of nodes with 1 child}$$

$$n_2 = \text{number of nodes with 2 children}$$

Then:

$$\boxed{n_0 = n_2 + 1}$$

This is one of the most important GATE properties.

**Practice 9 – NAT**

A binary tree has height 5 edges. What is the maximum possible number of nodes?

**Detailed Solution**

$$2^{h+1} - 1$$

$$= 2^6 - 1$$

$$= \boxed{63}$$

**Practice 10 – NAT**

A full binary tree contains 18 internal nodes. How many leaves does it contain?

**Detailed Solution**

For a full binary tree:

$$L = I + 1$$

Therefore:

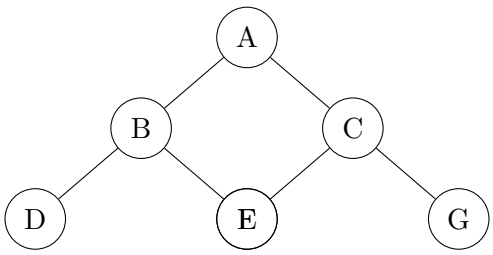
$$L = 18 + 1 = \boxed{19}$$

# 11 Binary Tree Traversals

The three fundamental depth-first traversals are:

1. Preorder
2. Inorder
3. Postorder

Consider:




## 11.1 Preorder

$$\boxed{Root \rightarrow Left \rightarrow Right}$$

For the above tree:

$$\boxed{A, B, D, E, C, F, G}$$

## 11.2 Inorder

$$\boxed{Left \rightarrow Root \rightarrow Right}$$

$$\boxed{D, B, E, A, F, C, G}$$

## 11.3 Postorder

$$\boxed{Left \rightarrow Right \rightarrow Root}$$

$$\boxed{D, E, B, F, G, C, A}$$

## 11.4 Level Order

Level-order traversal visits nodes level by level.

It uses a queue.

$$\boxed{A, B, C, D, E, F, G}$$

**Important**

Preorder begins with the root.
Postorder ends with the root.
Inorder of a BST produces sorted order.

# 12 Tree Reconstruction

A very important GATE question type is reconstructing a tree from traversal orders.

## 12.1 Preorder + Inorder

Preorder:

$$P, Q, S, E, R, F, G$$

Inorder:

$$S, Q, E, P, F, R, G$$

First element of preorder is:

$$P$$

Therefore $P$ is root.

In inorder:

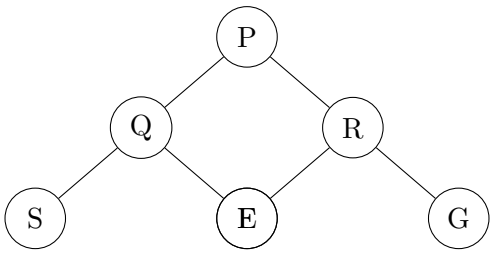
$$S, Q, E \quad P \quad F, R, G$$

Therefore:

$$\text{Left subtree} = S, Q, E$$

$$\text{Right subtree} = F, R, G$$

The final tree is:




Postorder:

$$\boxed{S, E, Q, F, G, R, P}$$

**Common GATE Trap**

Preorder + postorder does not uniquely determine a general binary tree.
However, for a full binary tree, the additional structure can remove the ambiguity.

# 13 Tree Practice

**Practice 11 – NAT**

A binary tree contains 31 nodes and is perfect. Find its height in edges.

**Detailed Solution**

$$31 = 2^{h+1} - 1$$

$$32 = 2^{h+1}$$

Therefore:

$$h + 1 = 5$$

$$\boxed{h = 4}$$

**Practice 12 – NAT**

A binary tree contains 25 nodes. Find the minimum possible height in edges.

**Detailed Solution**

Maximum nodes for height $h$:

$$2^{h+1} - 1$$

We need:

$$2^{h+1} - 1 \geq 25$$

For $h = 3$:

$$15 < 25$$

For $h = 4$:

$$31 \geq 25$$

Therefore:

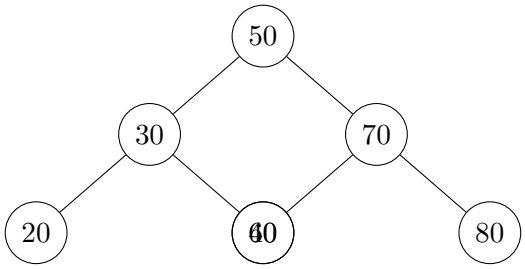
$$\boxed{4}$$

# 14 Binary Search Trees

A Binary Search Tree (BST) is a binary tree satisfying:

$$\boxed{\text{Left subtree keys} < \text{Root key} < \text{Right subtree keys}}$$

assuming distinct keys.




Inorder traversal:

$$\boxed{20, 30, 40, 50, 60, 70, 80}$$

Therefore:

$$\boxed{\text{Inorder traversal of a BST is sorted}}$$

## 14.1 BST Search

To search for a key $x$:

1. Compare $x$ with root.
2. If equal, found.
3. If smaller, move left.
4. If larger, move right.

Complexity:

$$\boxed{O(h)}$$

where $h$ is tree height.
Balanced BST:

$$h = O(\log n)$$

Therefore:

$$\boxed{O(\log n)}$$

Worst-case skewed BST:

$$h = n - 1$$

Therefore:

$$\boxed{O(n)}$$

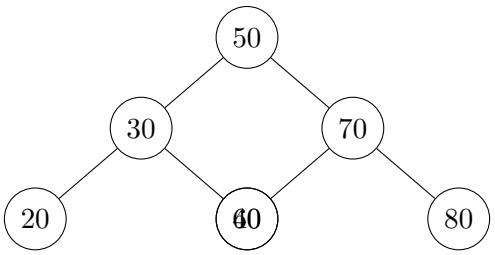
# 15 BST Insertion and Deletion

## 15.1 Insertion

Insert:

$$50, 30, 70, 20, 40, 60, 80$$

The resulting tree is:




## 15.2 BST Deletion

There are three cases.

### 15.2.1 Case 1: Leaf

Simply remove the node.

### 15.2.2 Case 2: One Child

Connect parent directly to the child.

### 15.2.3 Case 3: Two Children

Use:

$$\boxed{\text{Inorder successor}}$$

or

$$\boxed{\text{Inorder predecessor}}$$

Inorder successor:

$$\boxed{\text{minimum key in right subtree}}$$

Inorder predecessor:

$$\boxed{\text{maximum key in left subtree}}$$

**Practice 13 – NAT**

In the BST containing $50, 30, 70, 20, 40, 60, 80$, delete 30 using the inorder successor method. What key replaces 30?

**Detailed Solution**

Right subtree of 30 contains:

40

The minimum value is:

$\boxed{40}$

Therefore 40 replaces 30.

# 16 AVL Trees

An AVL tree is a self-balancing Binary Search Tree.

For every node:

$$\boxed{|BF| \leq 1}$$

where:

$$BF = height(left) - height(right)$$

Therefore:

$$BF \in \{-1, 0, +1\}$$

## 16.1 Four Rotation Cases

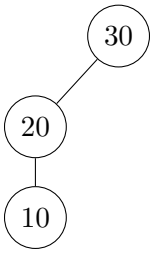
| Case | Rotation |
|---|---|
| LL | Right Rotation |
| RR | Left Rotation |
| LR | Left Rotation + Right Rotation |
| RL | Right Rotation + Left Rotation |

## 16.2 LL Case

Insert:

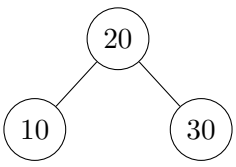
$$30, 20, 10$$

Tree becomes:




Apply right rotation.

Final:




Root:

$$\boxed{20}$$

### 16.3 RR Case

Insert:

$$10, 20, 30$$

Apply left rotation.

Final root:

$$\boxed{20}$$

### 16.4 LR Case

Insert:

$$30, 10, 20$$

First:

$$\boxed{\text{Left rotation at } 10}$$

Then:

$$\boxed{\text{Right rotation at } 30}$$

### 16.5 RL Case

Insert:

$$10, 30, 20$$

First:

$$\boxed{\text{Right rotation at } 30}$$

Then:

$$\boxed{\text{Left rotation at } 10}$$

## 17 AVL Complexity

AVL trees maintain logarithmic height.

Therefore:

$$\boxed{Search = O(\log n)}$$

$$\boxed{Insertion = O(\log n)}$$

$$\boxed{Deletion = O(\log n)}$$

The minimum number of nodes in an AVL tree satisfies:

$$N(h) = N(h-1) + N(h-2) + 1$$

This recurrence is Fibonacci-like and results in logarithmic height.

**Important**

An AVL tree is always a BST, but an arbitrary BST is not necessarily an AVL tree.

**Practice 14 – MCQ**

Which statement is correct?

1. Every BST is an AVL tree.
2. Every AVL tree is a BST.
3. AVL trees are not binary trees.
4. AVL trees can have arbitrary height for fixed $n$.

**Answer**

B

# 18 Heaps

A binary heap is a complete binary tree satisfying a heap-order property.

## 18.1 Max Heap

For every node:

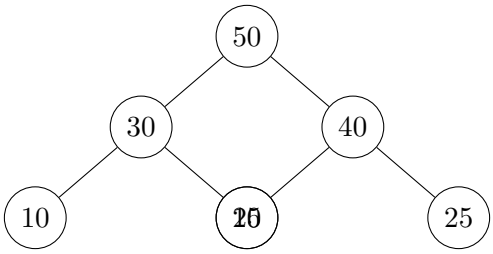
$$\boxed{Parent \geq Child}$$

## 18.2 Min Heap

For every node:

$$\boxed{Parent \leq Child}$$

Example max heap:




## 18.3 Heap Array Representation

For a 0-indexed array:

$$\boxed{left(i) = 2i + 1}$$

$$\boxed{right(i) = 2i + 2}$$

$$\boxed{parent(i) = \left\lfloor \frac{i-1}{2} \right\rfloor}$$

For a 1-indexed array:

$$left(i) = 2i$$

$$right(i) = 2i + 1$$

$$parent(i) = \left\lfloor \frac{i}{2} \right\rfloor$$

### 18.4 Heap Complexity

| Operation | Complexity |
|---|---|
| Find maximum/minimum | $O(1)$ |
| Insert | $O(\log n)$ |
| Extract maximum/minimum | $O(\log n)$ |
| Build heap | $O(n)$ |

**Common GATE Trap**

Building a heap using bottom-up heapify is $O(n)$, not $O(n \log n)$.

# 19 Heap Practice

**Practice 15 – NAT**

In a 0-indexed heap, what are the indices of the children of node at index 7?

**Detailed Solution**

Left:

$$2(7) + 1 = 15$$

Right:

$$2(7) + 2 = 16$$

Therefore:

$$\boxed{15, 16}$$

**Practice 16 – NAT**

A complete binary tree contains 15 nodes. What is its height in edges?

**Detailed Solution**

15 nodes form a perfect binary tree:

$$15 = 2^4 - 1$$

Therefore there are 4 levels and:

$$\boxed{3}$$

edges in the longest root-to-leaf path.

# 20 Hash Tables

A hash table stores keys using a hash function.

$$h : U \to \{0, 1, \ldots, m-1\}$$

where:

- $U$ = universe of keys,
- $m$ = table size.

## 20.1 Hash Function

A simple hash function:

$$\boxed{h(k) = k \bmod m}$$

Example:

$$m = 10$$

For:

$$k = 37$$

we get:

$$h(37) = 37 \bmod 10 = 7$$

## 20.2 Collision

A collision occurs when two different keys map to the same table position.
For example:

$$h(23) = 3$$

and:

$$h(43) = 3$$

Therefore 23 and 43 collide.

$$\boxed{\text{Collision}}$$

# 21 Load Factor

Load factor is:

$$\boxed{\alpha = \frac{n}{m}}$$

where:

- $n$ = number of stored keys,
- $m$ = number of slots.

Example:

$$n = 75, \quad m = 100$$

Then:

$$\alpha = \frac{75}{100}$$

$$\boxed{\alpha = 0.75}$$

**Common GATE Trap**

Do not confuse $\frac{n}{m}$ with $\frac{m}{n}$.

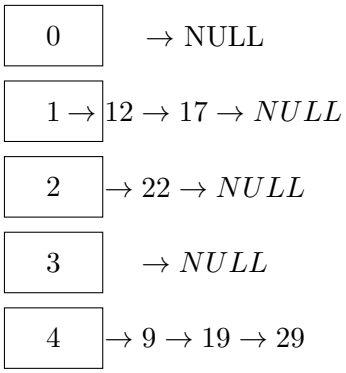
# 22 Collision Resolution

Two major techniques are:

1. Separate chaining
2. Open addressing

## 22.1 Separate Chaining

Each table position stores a chain/list.
Example:




Chaining can have:

$$\alpha > 1$$

because multiple keys can occupy the same slot through chains.

## 22.2 Open Addressing

In open addressing, all keys are stored directly inside the table.
Therefore:

$$\boxed{\alpha < 1}$$

is necessary for an available empty slot.
Common methods:

1. Linear probing
2. Quadratic probing
3. Double hashing

## 23 Linear Probing

Linear probing uses:

$$\boxed{h(k,i) = (h(k) + i) \bmod m}$$

where:

$$i = 0, 1, 2, \ldots$$

Example:

$$m = 10$$

$$h(k) = k \bmod 10$$

Insert:

$$23, 43, 13$$

For 23:

$$h(23) = 3$$

So:

$$23 \to 3$$

For 43:

$$h(43) = 3$$

Slot 3 is occupied.

Try:

$$4$$

Therefore:

$$43 \to 4$$

For 13:

$$h(13) = 3$$

Slots 3 and 4 are occupied.

Try slot 5.

Therefore:

$$13 \to 5$$

Final table:

$$3 : 23, \quad 4 : 43, \quad 5 : 13$$

## 23.1 Primary Clustering

Linear probing creates:

$$\boxed{\text{Primary Clustering}}$$

because occupied consecutive slots tend to form clusters.

**Important**

Insertion order matters in open addressing.